

CS383 - Software Engineering

Software Architecture (I)

Software architecture



- Describes major system's components and how they communicate, interoperate and exchange information
- Since it describes sub-systems
 - Ideally should be done at early stage of system design
 - Like between requirement specification and design process

Software architecture (cont.)



- There are two level of software architecture
 - Internal architecture: concerned about the decomposition of a program into components
 - External architecture: concerned about the architecture of complex systems (systems with many modules of programs) distribution over different computers of maybe beyond geographical boundary

Software architecture (cont.)



- Why software architecture?
 - Stakeholder communication: used as a focus discussion about the target system and its internal and external components
 - System analysis: helps to study whether the system meets its non-functional requirements
 - Large-scale reuse: for systems with similarities, software architecture can tell whether reuse is possible

Software architecture (cont.)

- Used symbols:
 - Rectangle box shape to represent components
 - Rectangle shape block contains a number of boxes to represent sub-components
 - Arrow lines to represent data flows or signals between components

Software architecture (cont.)

- Used symbols drawbacks:
 - Lack of semantics
 - E.g. arrows don't show relationships between entities
 - Very abstract

Architecture design and nonfunctional requirements

- 
- There is relationship between software architecture and non-functional requirements

Performance: critical operations should be localized (framed) within small components. The aim should be reduce the number of communication

Security: layered structure design should be used with innermost layers contain critical profile assets

Architecture design and nonfunctional requirements



Safety: safety-related operation should be all located in a single component or in localized number of components. This reduce the costs and problems of validation

Availability: redundant components should included in architecture design so that it is possible to update/replace component without stopping the system.

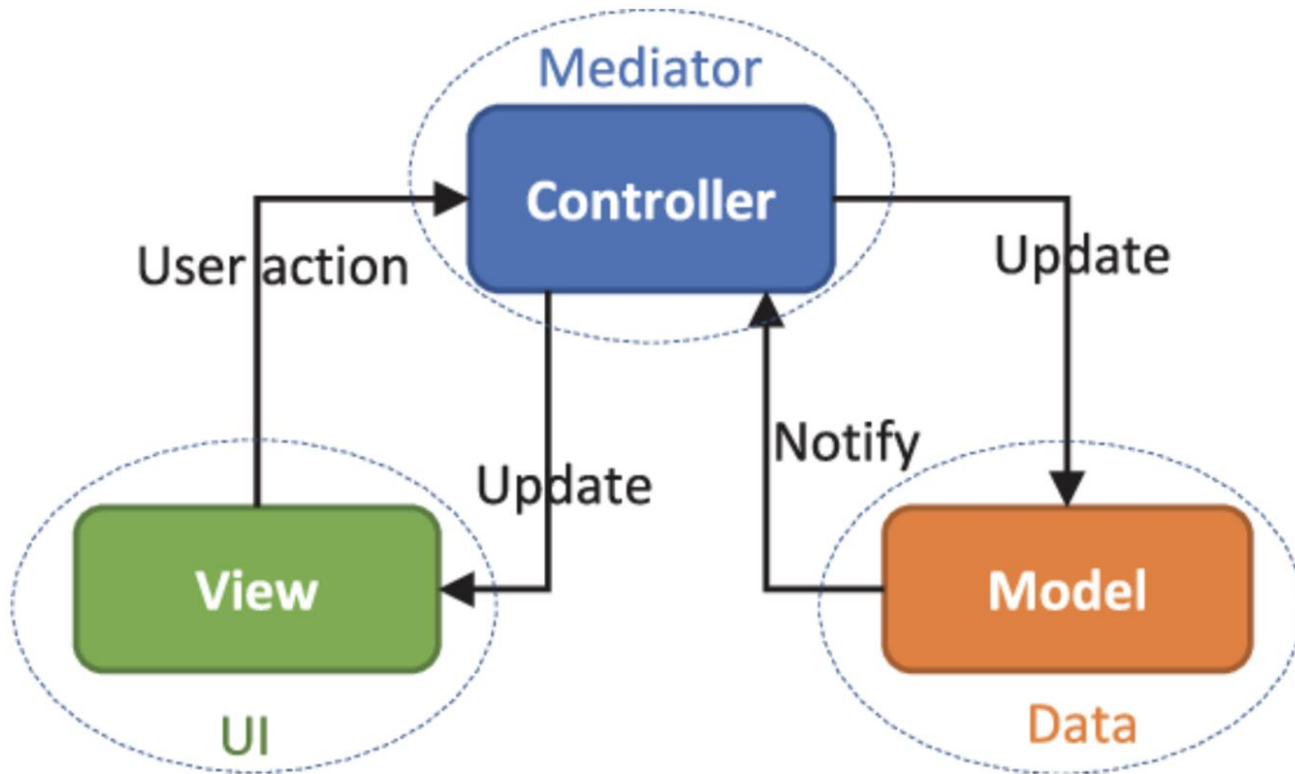
Maintainability: components that are need to be ready to change should be designed using fine-grain and they should be self-contained.

Architectural patterns

- A way of presenting, sharing and reusing knowledge about software system
- Widely used patterns are
 1. Model-View-Controller architecture
 2. Layered architecture
 3. Repository architecture
 4. Client-server architecture
 5. Pipe and filter architecture
 6. Service-oriented architecture

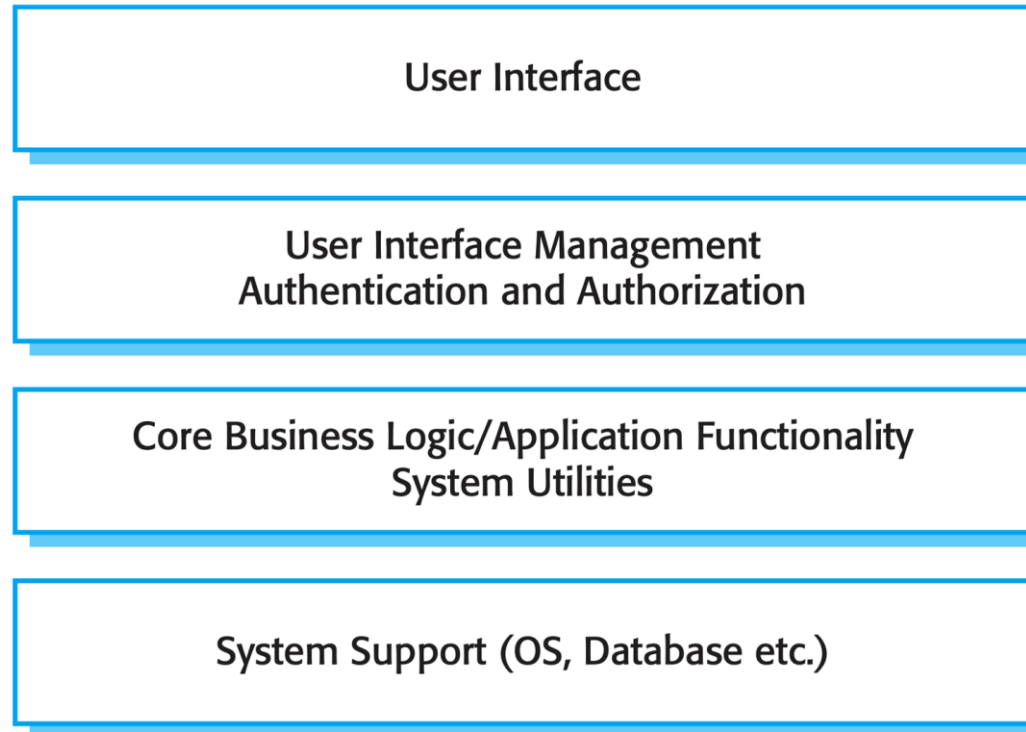
MVC architecture

- Mostly used in web-based development frameworks: ASP.NET, Ruby on Rails, Django



Layered architecture

- Mostly used for service-oriented services applications and SaaS.

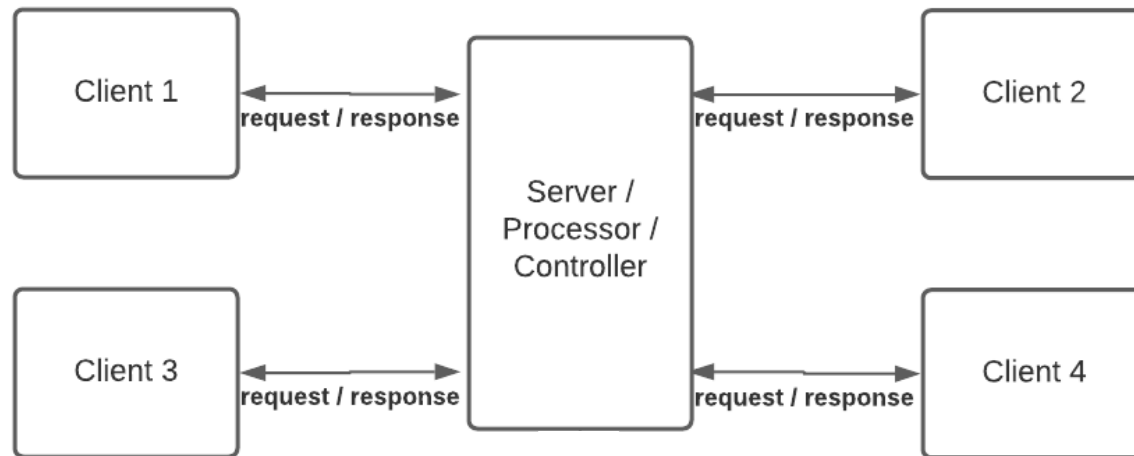


Repository architecture

- Describes how system components share data
- Examples: DBMS, and modern compilers
- There are two data exchange
 - Shared data: a central database that may be accessed by all components
 - Self-contained database that is only accessed by the contained component

Client-server architecture

- Mostly used for general access to static content on servers.
- Example: email server, file server, version control (such as SVN, Git)



Pipe and filter architecture

- Mostly used for applications that have chain execution
- This architecture can be used for applications that are developed using component-based (reuse-oriented) development model.
- The output of one component can be filtered/transformed or selected to be input for the next component.

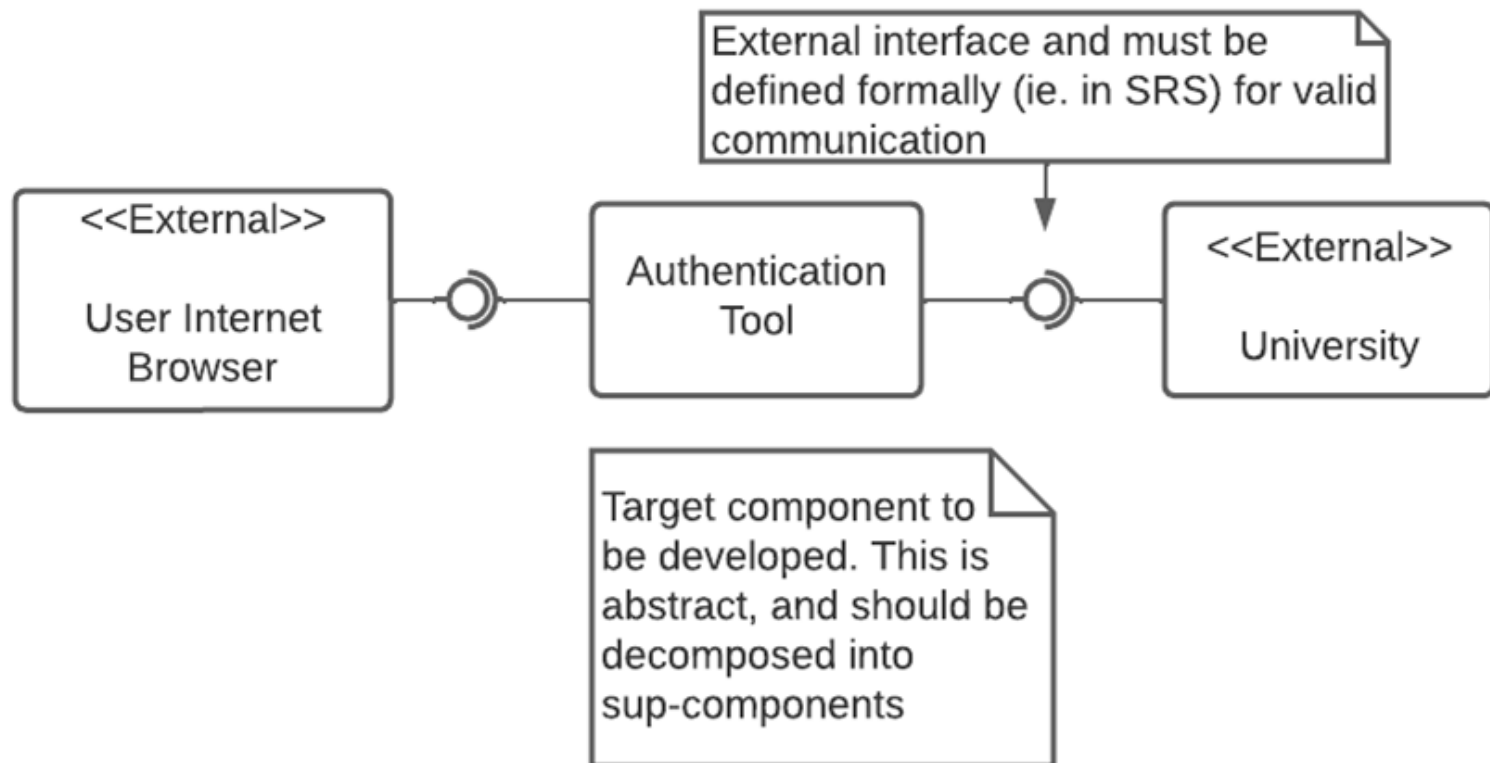


Software Architecture Example 1



A authentication tool is required to allow students of a university to access, via web-browsers, a cloud application. A student needs to submit his/her student credentials to the tool for access. The tool then checks the credentials with the university with pre-defined interface. If authenticated, the student is allowed to access the cloud application.

Software Architecture Example 1



Software Architecture Example 2

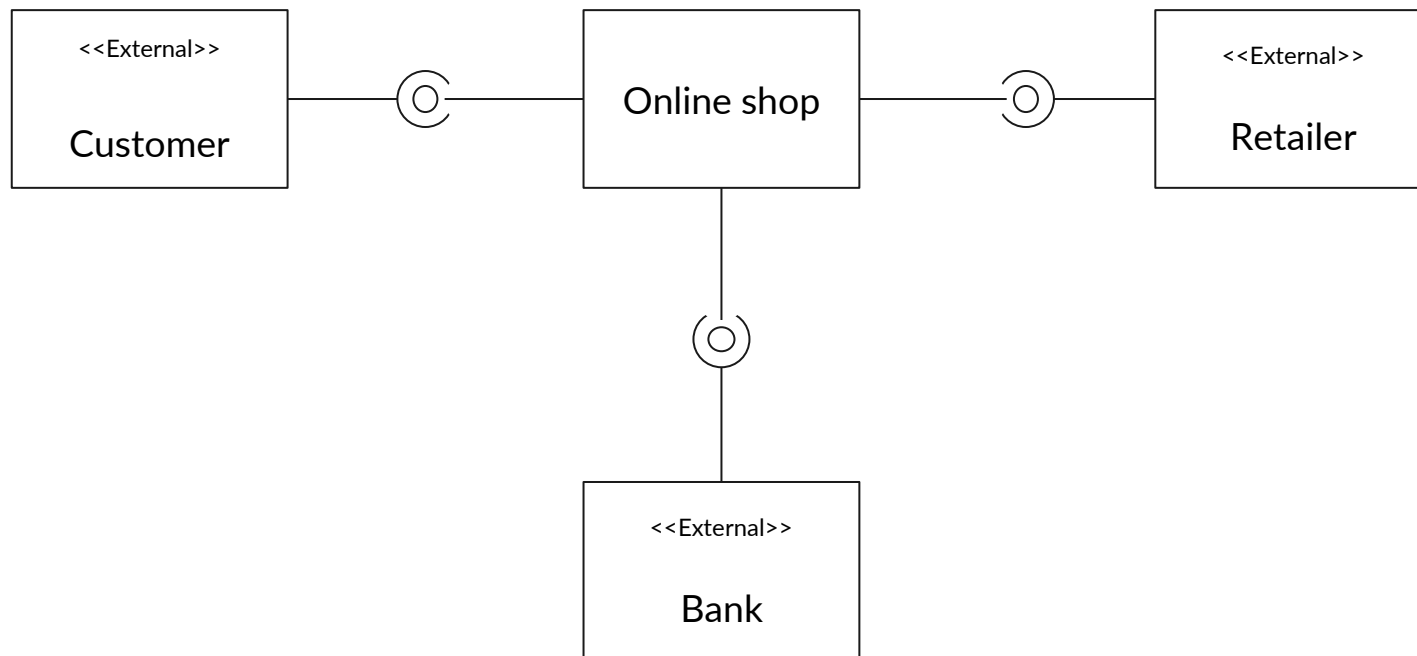


Clients wish to have a system that helps them to manage their online shop.

The shop allow visitors to explore online list of items or search for particular items. On selection of an item, the visitors can specify the desire quantity and add them to the shopping cart. On checkout, the visitor can specify shipping address and payment details (or load if registered customer). Once approved transaction of payment with the bank, the details of the shipments are sent to the target retailers of the items.

Draw a software architectural design that shows the main components of the given description above.

Software Architecture Example 2



Software Architecture Example 3

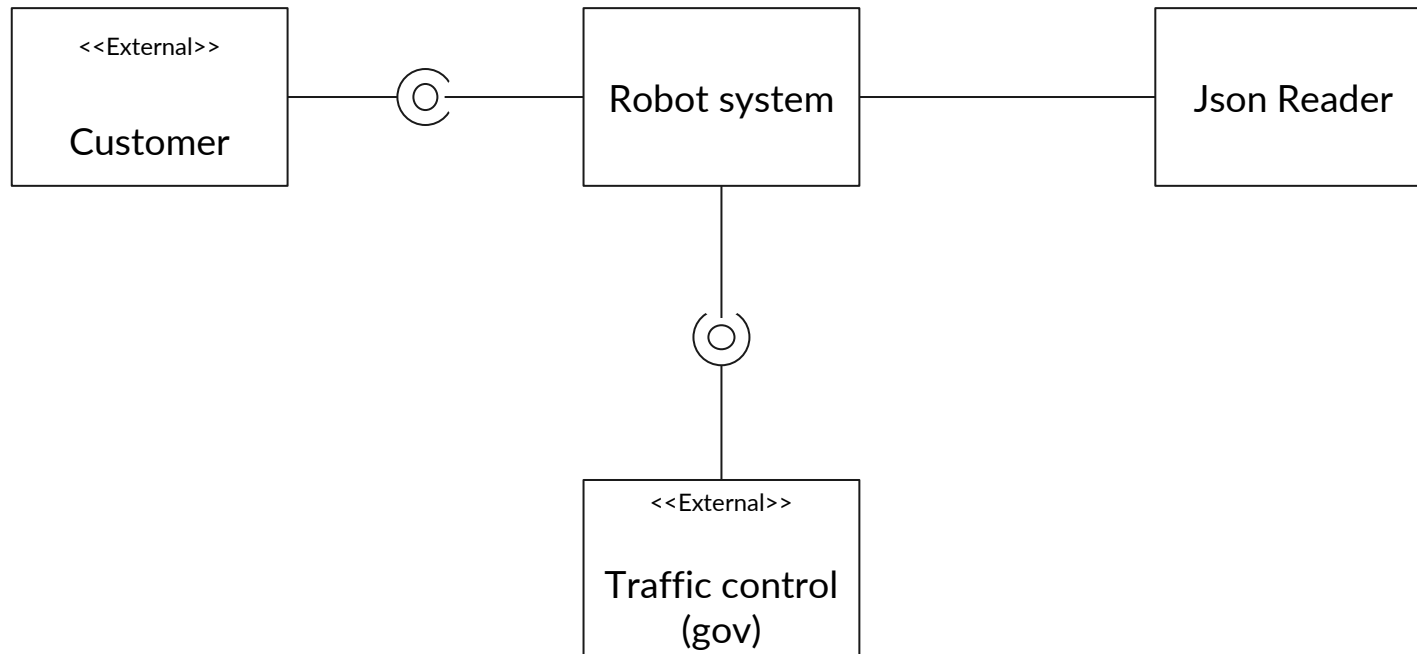
A robot delivery machine works daily to deliver shipments to customers.

Customers locations are provided as a JSON document that is processed by the robot and the delivery priority is planned based on the customer locations and the dynamic road traffic obtained from the government.

Before delivery, the customer receives an automated text message and a notification of the delivery.

Draw a software architectural design that shows the main components of the given description above.

Software Architecture Example 3



Software Architecture Example 4



A train transportation company have an autonomous train system. The train stops for 10 minutes only at designated and rented platforms to allow customers to leave or get on the train. Not all trains can use same platforms.

Customer can use a QR code to open the train doors only while the train is at stop position. Buying tickets is done via dedicated mobile app that is connected externally with the train system. The train is always observed by an external operator by the company.

Draw a software architectural design that shows the main components of the given description above.

Software Architecture Example 4

